

Matrix Sketching and Linear Programming

by

Jamil Zhumabek

Algorithms for Big Data

Course Project

Mohamed bin Zayed University of Artificial Intelligence

October 2026

This report is adapted from the author's M.Sc. thesis, *Matrix Sketching and Linear Programming*, Nazarbayev University, 2025. It retains the part on random projections for linear programming and presents a fresh, reduced experimental evaluation with a Netlib benchmark extension.

Abstract

Random projection has become a standard tool for dimensionality reduction, yet research is still being conducted on its efficiency for large linear programs. This project studies the random-projection approach to linear programming, develops the relevant theoretical background, and evaluates the method through numerical experiments.

After reviewing the Johnson–Lindenstrauss lemma, we prove that if the original linear programming problem is unbounded, then the projected one is unbounded as well. A counterexample shows that the converse is not necessarily true. The forward implication holds for every linear projection because the projected constraints relax the original ones.

We conduct numerical experiments on synthetically generated feasible and infeasible instances, and on five established Netlib LP benchmarks. The experiments compare running time, equality residuals, nonnegativity violations, and objective discrepancies. They distinguish a solution of the projected problem from a feasible solution of the original problem. In particular, reducing the number of constraints does not by itself guarantee either a computational improvement or a valid recovered solution. The Netlib extension also measures matrix storage and checks unbounded-ray certificates. It evaluates the basic constraint relaxation, not sketching methods for optimization in general.

Chapter 1

Introduction

1.1 Motivation and Background

Matrix sketching is a technique designed to efficiently approximate a large matrix using a more compact representation. The goal is to preserve properties of interest within a guaranteed approximation ratio. Random projections achieve this by randomly combining rows of the original matrix. The Johnson–Lindenstrauss lemma provides a fundamental guarantee for preserving distances between a finite set of points in a lower dimensional space.

In this work we concentrate particularly on the random projections method with application to linear programming problems. Linear programming is widely applied in operations research, machine learning, economics, and combinatorics. However, a large constraint matrix can make its solution computationally expensive. A smaller projected matrix provides a possible way of reducing this cost.

Our research problem is to understand what is retained when the equality constraints of a linear program are projected, and whether the reduction in problem size improves computational performance. We follow the random-projection formulation studied by Vu, Poirion, and Liberti [1]. This report focuses on its basic projected relaxation and the two recovery procedures described in the author’s thesis [5]. It does not claim the full approximation guarantees of [1], which require additional assumptions and restrictions on the projected problem.

1.2 Relationship to Previous Work

The theoretical starting point is the author’s M.Sc. thesis, *Matrix Sketching and Linear Programming* [5]. The earlier experimental implementation is available in the companion GitHub project [6]. This course project retains the relevant theoretical material and the two recovery strategies, while replacing the earlier experimental chapter with newly executed experiments. The unboundedness result is retained from the thesis; no claim of a new literature contribution is made here.

The previous repository is cited for provenance and is not modified. The revised code, recorded seeds, raw observations, environment metadata, and report source are supplied in the separate course-project repository `random-projections-lp-course-project`. The master’s thesis PDF and previous notebook outputs are not part of this package.

1.3 Research Objectives

The primary objectives of this project are as follows:

- 1) To introduce random projections and the theoretical background needed for their application to linear programming.
- 2) To prove the preservation of unboundedness under projection and show that the converse does not necessarily hold.
- 3) To conduct numerical experiments on feasible and infeasible instances and established LP benchmarks, comparing solution quality, running time, and projected matrix storage.

1.4 Matrix Sketching and Random Projections

Let A be an $m \times n$ constraint matrix. We consider the following linear programming problem:

$$\begin{aligned} (P) \quad & \text{minimize} && c^\top x \\ & \text{subject to} && Ax = b, \\ & && x \geq 0, \end{aligned} \tag{1.1}$$

where $c \in \mathbb{R}^n$ and $b \in \mathbb{R}^m$. Let S be a short and fat $k \times m$ matrix, where $k < m$. For a Gaussian random projection we take independent entries $S_{ij} \sim \mathcal{N}(0, 1/k)$. The matrix product has dimensions

$$\underbrace{S}_{k \times m} \underbrace{A}_{m \times n} = \underbrace{SA}_{k \times n}.$$

Then, we solve the projected problem:

$$\begin{aligned} (P_S) \quad & \text{minimize} && c^\top x \\ & \text{subject to} && SAx = Sb, \\ & && x \geq 0. \end{aligned} \tag{1.2}$$

The number of equality constraints decreases from m to k , while the number of variables remains n . We compare solution quality and running time between the original and the sketched problem.

1.5 Other Sketching Methods and Their Relevance

The thesis surveys several sketching techniques before specializing to linear programming [5, pp. 6–16]. We retain a focused part of that background here. For a row sketch $B = SA$, one possible quality criterion is the covariance error $\|A^\top A - B^\top B\|_2$. For a unit vector z , this controls the error in $\|Az\|_2^2$. Such a criterion concerns matrix geometry; by itself it does not certify the feasibility or boundedness of a projected LP.

1.5.1 Row Sampling and Sparse Embeddings

Let $a_i^\top$ be row i of A , with $A \neq 0$. Norm-based row sampling draws k independent indices with probabilities $p_i = \|a_i\|_2^2 / \|A\|_F^2$ and takes the sampled rows $a_i^\top / \sqrt{k p_i}$. Zero rows are

never sampled. The normalization gives

$$\mathbb{E}[B^\top B] = \sum_{j=1}^k \sum_{i:p_i>0} p_i \frac{a_i a_i^\top}{k p_i} = A^\top A.$$

Unbiasedness is not a uniform approximation guarantee, and leverage-based sampling is a different choice of sampling distribution.

CountSketch instead hashes each original row to a bucket $h(i) \in \{1, \dots, k\}$ and assigns an independent random sign $s_i \in \{-1, 1\}$. Its sketching matrix has exactly one nonzero per column:

$$S_{ri} = s_i \mathbf{1}\{h(i) = r\}, \quad B_{r,:} = \sum_{i:h(i)=r} s_i A_{i,:}.$$

For a fixed vector v , independent mean-zero signs give $\mathbb{E} \|Sv\|_2^2 = \|v\|_2^2$. Stronger embedding guarantees require dimension and probability conditions [7]; this expectation alone is not the JL lemma. Accumulating the contributions can take time proportional to $\text{nnz}(A)$, apart from allocation and storage management. Collisions may cancel entries or leave empty sketch rows. The Netlib experiments test CountSketch alongside Gaussian and sparse-valued random projections, using actual sparse storage.

1.5.2 Low-Rank Approximation and Streaming Sketches

Randomized range finding draws $\Omega \in \mathbb{R}^{n \times \ell}$, computes $Y = A\Omega$, and obtains an orthonormal basis Q for its range. A factorization of the smaller matrix $Q^\top A$ then approximates a factorization of A [8]. This data-dependent low-rank approximation is distinct from replacing $Ax = b$ by an oblivious row sketch. Frequent Directions is a deterministic streaming approach that shrinks singular values while maintaining a small covariance sketch [9]. These methods provide context from the thesis; neither is implemented or experimentally evaluated in this project.

1.5.3 What Would Preserve All Equality Residuals?

Suppose that S is a subspace embedding for $\mathcal{V} = \text{range}([A \ b])$ with $0 < \epsilon < 1$, so that

$$(1 - \epsilon) \|v\|_2^2 \leq \|Sv\|_2^2 \leq (1 + \epsilon) \|v\|_2^2 \quad (v \in \mathcal{V}).$$

Every residual $Ax - b$ belongs to $\mathcal{V}$. The lower bound implies $S(Ax - b) = 0$ if and only if $Ax - b = 0$. Thus this stronger condition would preserve all equalities exactly. However, it requires S to be injective on $\mathcal{V}$ and hence $k \geq \dim \mathcal{V}$. If A has full row rank, $\mathcal{V} = \mathbb{R}^m$ and no sketch with $k < m$ can satisfy it. This elementary observation explains why preserving a finite set of distances cannot automatically justify preservation of an entire LP feasible set. It is not a claimed new theorem.

1.5.4 Sketching as a Preconditioner

The thesis also reviews randomized interior-point methods [5, pp. 31–35]. Their computational difficulty includes repeated linear systems involving a normal matrix $M = AD^2A^\top$, where D is a positive diagonal scaling. A suitable sketch W can form $\widetilde{M} = ADWW^\top DA^\top$ as a preconditioner for an iterative solve. In this use, sketching accelerates a linear-algebra subproblem; it does not simply discard equality constraints from the optimization problem. The analysis of Chowdhury et al. includes safeguards for the inexact solves [10]. We do not implement that algorithm. Consequently, a failure of our basic relaxation is not evidence against randomized preconditioning or all sketch-based LP methods.

Chapter 2

Main Result

This chapter establishes the theoretical background for analyzing linear programming problems under random projections. The Johnson–Lindenstrauss lemma explains the geometric motivation for dimensionality reduction. Farkas’ lemma gives a certificate of infeasibility. We then prove preservation of unboundedness, retaining the result and counterexample developed in the thesis [5].

2.1 Preliminaries

Lemma 2.1.1 (The Johnson–Lindenstrauss lemma). *Let $V \subset \mathbb{R}^m$ be a set of $N \geq 2$ points and let $0 < \epsilon < 1$. For every integer*

$$k \geq \frac{4 \ln N}{\epsilon^2/2 - \epsilon^3/3}, \tag{2.1}$$

there exists a linear map $f : \mathbb{R}^m \rightarrow \mathbb{R}^k$ such that, for all $u, v \in V$,

$$(1 - \epsilon) \|u - v\|_2^2 \leq \|f(u) - f(v)\|_2^2 \leq (1 + \epsilon) \|u - v\|_2^2.$$

Proof. Our aim is to estimate the length of a fixed vector after it has been mapped into a lower dimensional space. We use the Gaussian concentration argument underlying elementary proofs of the lemma [2, 3].

Let the entries of $S \in \mathbb{R}^{k \times m}$ be independent $\mathcal{N}(0, 1/k)$ variables, and define $f(w) = Sw$. For a fixed $w \neq 0$, the random variable

$$Z = \frac{k \|Sw\|_2^2}{\|w\|_2^2}$$

is the sum of squares of k independent standard normal variables. Its moment generating function is

$$\mathbb{E}[e^{tZ}] = (1 - 2t)^{-k/2}, \quad t < \frac{1}{2}.$$

Applying Markov's inequality to e^{tZ} and minimizing the resulting bound over $t > 0$, with $t = \epsilon/[2(1 + \epsilon)]$, gives

$$\begin{aligned} \Pr[Z \geq k(1 + \epsilon)] &\leq \exp\left[-\frac{k}{2}(\epsilon - \ln(1 + \epsilon))\right] \\ &\leq \exp\left[-\frac{k}{2}\left(\frac{\epsilon^2}{2} - \frac{\epsilon^3}{3}\right)\right]. \end{aligned}$$

The last step follows from $\ln(1 + \epsilon) \leq \epsilon - \epsilon^2/2 + \epsilon^3/3$. Similarly, applying Markov's inequality to e^{-tZ} and choosing $t = \epsilon/[2(1 - \epsilon)]$ gives

$$\begin{aligned} \Pr[Z \leq k(1 - \epsilon)] &\leq \exp\left[\frac{k}{2}(\epsilon + \ln(1 - \epsilon))\right] \\ &\leq \exp(-k\epsilon^2/4). \end{aligned}$$

Under (2.1), each tail probability is at most N^{-2} . Apply these estimates to each nonzero difference $u - v$, and then use the union bound over the $\binom{N}{2}$ pairs. The probability that any pair violates the stated inequalities is at most

$$\binom{N}{2} \frac{2}{N^2} = 1 - \frac{1}{N} < 1.$$

Hence, there exists a mapping with the required properties. $\square$

The lemma guarantees distance preservation for a finite set of vectors. It does not automatically guarantee that all feasible points, all recession directions, or the optimal value of an LP are preserved. The analysis of [1] accounts for additional geometric conditions. Our experiments use $k = \lfloor m/2 \rfloor$ as a fixed compression level; this choice is empirical and is not asserted to satisfy (2.1) for a prescribed ϵ .

Lemma 2.1.2 (Farkas' lemma). *Exactly one of the following two systems has a solution:*

$$Ax = b, \quad x \geq 0; \quad \text{or} \quad A^\top y \geq 0, \quad b^\top y < 0.$$

This is the standard equality-form alternative for linear programming [4]. The two systems cannot both hold: if they did, then

$$b^\top y = (Ax)^\top y = x^\top A^\top y \geq 0,$$

contradicting $b^\top y < 0$. We use this certificate to generate infeasible instances in Chapter 3.

Definition 2.1.3 (Recession direction). For a nonempty standard-form polyhedron $F = \{x \in \mathbb{R}^n : Ax = b, x \geq 0\}$, a recession direction is a vector d satisfying $Ad = 0$ and $d \geq 0$. Then $x + td \in F$ for every $x \in F$ and $t \geq 0$.

2.2 Preservation of Unboundedness

Theorem 2.2.1. *Assume the original LP*

$$\min\{c^\top x : Ax = b, x \geq 0\}$$

is unbounded below. Then for every matrix $S \in \mathbb{R}^{k \times m}$ the projected LP

$$\min\{c^\top x : S Ax = S b, x \geq 0\}$$

is unbounded below as well. The converse is not necessarily true.

Proof. Since the original LP is unbounded, there exists a feasible point x_0 and a recession direction d such that

$$Ax_0 = b, \quad x_0 \geq 0, \quad Ad = 0, \quad d \geq 0, \quad c^\top d < 0.$$

This is the standard recession characterization of unboundedness for a nonempty linear program [4]. To construct an unbounded ray, define for any $t \geq 0$

$$x(t) = x_0 + td.$$

Since $Ad = 0$, substituting into the projected constraints gives

$$SA(x_0 + td) = SAx_0 + tSAd = Sb.$$

Also, $x_0 + td \geq 0$. In other words, the same ray is feasible for the projected problem. The objective remains unchanged:

$$c^\top(x_0 + td) = c^\top x_0 + tc^\top d,$$

which tends to $-\infty$ as $t \rightarrow \infty$. Hence, the projected LP is unbounded. No condition on the distribution of S is required because $Ad = 0$ implies $SAd = 0$ identically.

Let us show that the converse does not hold using the following counterexample. For the original LP, let

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad c = \begin{pmatrix} 0 \\ -1 \end{pmatrix}.$$

The solution of $Ax = b$ is $x = (1, 2)^\top$. Thus, the feasible set is a single point and the original optimal value is -2 . For the projected LP, let

$$S = \begin{pmatrix} 1 & 0 \end{pmatrix}.$$

Then $SA = (1 \ 0)$ and $Sb = 1$. The only equality is $x_1 = 1$, while $x_2 \geq 0$ is free to increase. The objective $c^\top x = -x_2$ tends to $-\infty$ as $x_2 \rightarrow \infty$. Thus, the projected LP is unbounded, yet the original LP is bounded. $\square$

The theorem can also be seen directly from inclusion of feasible sets: projection relaxes the equalities while keeping the objective and nonnegativity constraints. In particular, $v(P_S) \leq v(P)$ whenever both optimal values are finite. This elementary observation is useful for interpreting the experiments; it is not a probabilistic consequence of the Johnson–Lindenstrauss lemma.

Chapter 3

Synthetic Experiments and Results

3.1 Experimental Setup

We considered the pairs $(m, n) = (100, 150), (250, 375), (500, 750)$, constraint matrix densities $\text{dens} \in \{0.1, 0.5\}$, and $k = \lfloor m/2 \rfloor$. For each configuration and each of the three instance families, we generated 10 independent original LPs. Entries of A were sampled uniformly on $[0, 1]$ and retained independently with probability dens ; other entries were set to zero. The objective vector was always $c = \mathbf{1}$.

- 1) *Feasible instances*: sample x_0 uniformly on $[0, 1]^n$ and set $b = Ax_0$.
- 2) *Legacy all-negative instances*: sample $b_i = -u_i$, independently, with $u_i \sim U[0, 1]$. This retains the right-hand-side distribution in the previous infeasible-instance notebook [6].
- 3) *Additional mixed-sign stress test*: sample each b_i uniformly on $[0.5, 1.5]$ and replace b_1 by $-b_1$. This is an additional instance family, not the original thesis generator.

For both infeasible families, $A^\top e_1 \geq 0$ and $b^\top e_1 < 0$ in every generated instance. Thus, Lemma 2.1.2 certifies infeasibility before solving. Both families contain an immediate contradiction in the original equalities, which presolve may detect.

This is a fresh execution of an explicitly revised protocol. The previous notebooks [6] were inspected at commit `3d2ff96`; the full commit identifier is recorded in the repository's

provenance file. Their saved configurations used larger matrices, different compression, and fixed-count nonzero selection. The updated experiment uses the smaller grid above and Bernoulli masks. For feasible instances, it uses the same explicit solver for the original and projected LPs, replacing the earlier combination of SciPy and CVXPY's automatic solver choice. Direct and SVD solves replace explicit matrix inversion and normal equations. Projector normalizations, random streams, and solver-status reporting are made explicit below. Therefore, these are new measurements of the stated protocol, not an exact reconstruction of the old timing tables. No earlier experimental measurements are reused in this chapter.

3.1.1 Random Projectors

For feasible instances, we used a Gaussian projector with independent entries $\mathcal{N}(0, 1/k)$. For infeasible instances, we used the following two projectors:

- 1) A sparse random projector, with independent entries

$$S_{ij} = \sqrt{\frac{3}{k}} \begin{cases} +1, & \text{with probability } 1/6, \\ 0, & \text{with probability } 2/3, \\ -1, & \text{with probability } 1/6. \end{cases}$$

This normalization gives $\mathbb{E}[S_{ij}^2] = 1/k$ [3].

- 2) An orthogonal random projector. We formed a Gaussian $m \times k$ matrix and computed a reduced QR decomposition, obtaining $Q^\top Q = I_k$. We set $S = \sqrt{m/k} Q^\top$.

The scaling factors are needed for the usual distance-preservation interpretation. Multiplying all of S by the same nonzero scalar leaves the equality system $S Ax = S b$ unchanged in exact arithmetic.

3.1.2 Recovery of a Primal Vector

We retained the following two recovery procedures from the thesis [5], using the original implementation [6] as a reference for the method. Both return a candidate vector that must be checked against the original constraints.

Algorithm 1: Exact-solve (Dual-based recovery)

- 1: Obtain a projected dual solution $\lambda^* \in \mathbb{R}^k$ and form

$$y_{\text{prox}} = S^\top \lambda^*, \quad z_j = \frac{c_j - A_j^\top y_{\text{prox}}}{\|A_j\|_2}.$$

Here λ^* is the SciPy equality marginal, whose sign convention gives $c - (SA)^\top \lambda^* \geq 0$. Zero column norms are assigned an infinite score, and ties are resolved by column index.

- 2: Select $B \subseteq \{1, \dots, n\}$ as the indices of the m smallest normalized reduced costs z_j .
- 3: If A_B is nonsingular, solve $A_B x_B = b$ and set $x_j = 0$ for $j \notin B$. A singular selected matrix is recorded as a recovery failure.

Algorithm 2: Pseudoinverse-solve (Primal-based recovery)

- 1: Obtain a projected primal solution $\bar{x} \in \mathbb{R}^n$.
- 2: Select H as the indices of the m largest entries of $\bar{x}$.
- 3: Compute $x_H = A_H^+ b$ and set $x_j = 0$ for $j \notin H$.

Here A_H^+ denotes the Moore–Penrose pseudoinverse. We use an SVD least-squares solve instead of explicitly forming $(A_H^\top A_H)^+ A_H^\top$, which is algebraically equivalent but can worsen numerical conditioning. Neither procedure enforces $x \geq 0$.

3.1.3 Solution Quality and Running Time

For a returned vector x , we use the thesis’s normalized metrics:

$$\begin{aligned} \text{feas}(x) &= \frac{\|Ax - b\|_1}{\|b\|_1}, \\ \text{neg}(x) &= \frac{\sum_{j: x_j < 0} |x_j|}{\|x\|_1}, \\ \text{obj}(x) &= \frac{|c^\top x - v(P)|}{|v(P)|}. \end{aligned}$$

We evaluate these metrics separately for the projected solution $\bar{x}$ and the two recovered vectors. For $\bar{x}$, the objective is the projected optimal value. For a recovered vector, $c^\top x$ is only a candidate objective value unless original feasibility has been checked. A small equality residual does not compensate for negative components. The generated right-hand

sides and the feasible reference optimal values are nonzero. The implementation leaves a metric undefined if its reference denominator vanishes, except that $\text{neg}(0) = 0$.

A candidate is accepted as feasible for the original LP only if

$$\|Ax - b\|_\infty \leq 10^{-7} \max(1, \|b\|_\infty), \quad \min_j x_j \geq -10^{-7}.$$

Timing tables give means and, for the feasible family, sample standard deviations over 10 instances. Quality tables give arithmetic means; counts refer to the same 10 instances. All sample standard deviations, solver messages, timing components, and seeds are supplied in the raw and summary files.

The experiments were executed on 13 September 2026 using Python 3.12.14, NumPy 2.5.3, SciPy 1.18.1, and HiGHS 1.12.0 on an ARM64 Mac with Apple Accelerate. We used HiGHS dual simplex (**highs-ds**), presolve, and primal and dual feasibility tolerances of 10^{-7} . Single-thread settings were requested for HiGHS and numerical libraries; Accelerate’s active thread count was not independently measured. Separate data and projector streams were derived from master seed 20260913. The two infeasible projectors share each original instance within each family. In total, there are 180 independent original LPs and 300 projected LPs.

We measured elapsed wall-clock time. Original time covers its solver call. Projected time includes sampling S , computing SA and Sb , and solving the projected LP. Each recovery total includes that common projected cost and its own recovery step. Data generation, metric evaluation, and diagnostic retries are excluded. All matrices, including the sparse-valued projector, were stored densely. This is therefore not a sparse-storage benchmark.

3.2 Feasible Instances

All original and projected feasible-family LPs were solved to optimality. Table 3.1 reports the time to solve the original problem, the projected relaxation, and each projection-and-recovery workflow. A shorter projected workflow must be interpreted together with the quality checks.

Table 3.1: Fresh feasible-instance timings in milliseconds (mean $\pm$ sample standard deviation; d : density). Projected totals include sampling, multiplication and solving; recovery totals add the indicated recovery.

m	d	Original	Projected	Dual total	Primal total
100	0.1	3.05 ± 0.29	2.86 ± 0.13	3.04 ± 0.19	3.56 ± 0.13
100	0.5	6.39 ± 0.22	2.85 ± 0.15	3.00 ± 0.15	3.55 ± 0.14
250	0.1	40.95 ± 1.64	25.40 ± 0.74	25.97 ± 0.77	28.65 ± 0.76
250	0.5	85.65 ± 4.31	26.99 ± 1.31	27.54 ± 1.31	30.26 ± 1.31
500	0.1	481.54 ± 37.79	206.13 ± 5.61	208.28 ± 5.61	217.63 ± 5.63
500	0.5	886.02 ± 65.74	217.90 ± 9.64	220.07 ± 9.67	229.43 ± 9.62

Table 3.2 reports original-system residuals, negative-mass ratios, and objective discrepancies. The projected solutions retain nonnegativity but do not satisfy the original equalities. The recovered vectors have very small equality residuals, but substantial negative components. Neither recovery procedure returns an accepted feasible solution in this benchmark. Accordingly, a recovery timing is not the time to obtain a valid solution of the original LP.

Table 3.2: Mean relative equality residual, negative mass ratio and relative objective gap (d : density; Raw: projected primal; Dual/Primal: Algorithms 1/2). Accepted counts require equality feasibility and nonnegativity.

m	d	Method	Eq. residual	Neg. mass	Obj. gap	Accepted
100	0.1	Raw	2.98×10^{-1}	0.000	0.209	0/10
100	0.1	Dual	3.52×10^{-14}	0.444	0.799	0/10
100	0.1	Primal	2.22×10^{-15}	0.417	0.236	0/10
100	0.5	Raw	1.36×10^{-1}	0.000	0.106	0/10
100	0.5	Dual	1.52×10^{-15}	0.446	0.066	0/10
100	0.5	Primal	4.94×10^{-14}	0.388	1.717	0/10
250	0.1	Raw	2.07×10^{-1}	0.000	0.163	0/10
250	0.1	Dual	2.76×10^{-14}	0.454	0.837	0/10
250	0.1	Primal	5.25×10^{-15}	0.432	0.303	0/10
250	0.5	Raw	8.40×10^{-2}	0.000	0.070	0/10
250	0.5	Dual	4.94×10^{-15}	0.451	0.083	0/10
250	0.5	Primal	2.57×10^{-15}	0.444	0.085	0/10
500	0.1	Raw	1.59×10^{-1}	0.000	0.126	0/10
500	0.1	Dual	8.80×10^{-14}	0.474	0.557	0/10
500	0.1	Primal	9.09×10^{-15}	0.458	0.280	0/10
500	0.5	Raw	6.50×10^{-2}	0.000	0.052	0/10
500	0.5	Dual	1.56×10^{-14}	0.465	0.074	0/10
500	0.5	Primal	2.11×10^{-15}	0.456	0.046	0/10

These observations are consistent with the recovery procedures: solving the selected linear system does not enforce nonnegativity. Moreover, a basic projected optimum has at most k positive entries. Algorithm 2 selects $m > k$ entries, so remaining columns may be selected among zero-valued ties. We resolve such ties by ascending column index, which can influence the recovered vector. This experiment identifies a limitation of these heuristics for the selected instance distributions and compression level, rather than a failure of every recovery method for random-projection LPs.

3.3 Infeasible Instances

Every original LP in both infeasible families was reported infeasible. We report primary projected statuses separately: I means infeasible; F means optimal and hence feasible for the projected LP; E means an unresolved solver error. A projected feasible status must not be interpreted as feasibility of the original LP.

3.3.1 Legacy All-Negative Family

Table 3.3 evaluates the original notebook’s all-negative right-hand-side distribution on the updated grid. All projected LPs in this family were reported infeasible. This is an empirical observation for the sampled instances, not a general preservation guarantee.

Table 3.3: Fresh all-negative-family results (legacy right-hand-side distribution). Here d denotes density; times are means in milliseconds. Counts are primary projected-solver outcomes; all original LPs are infeasible.

m	d	Projector	Orig. (ms)	Proj. (ms)	Inf.	Opt.	Error
100	0.1	Sparse	0.64	1.95	10	0	0
100	0.1	Orthogonal	0.64	2.08	10	0	0
100	0.5	Sparse	1.04	1.71	10	0	0
100	0.5	Orthogonal	1.04	1.95	10	0	0
250	0.1	Sparse	1.55	12.46	10	0	0
250	0.1	Orthogonal	1.55	13.35	10	0	0
250	0.5	Sparse	4.61	10.97	10	0	0
250	0.5	Orthogonal	4.61	12.02	10	0	0
500	0.1	Sparse	4.51	67.39	10	0	0
500	0.1	Orthogonal	4.51	69.31	10	0	0
500	0.5	Sparse	18.63	65.71	10	0	0
500	0.5	Orthogonal	18.63	64.72	10	0	0

3.3.2 Additional Mixed-Sign Stress Test

Table 3.4 shows a different outcome for the additional mixed-sign family. The sparse projector returned 50 infeasible, 8 optimal, and 2 unresolved outcomes; the orthogonal projector returned 49 infeasible, 9 optimal, and 2 unresolved outcomes. Thus, some projected relaxations are feasible even though their original LPs have an explicit Farkas certificate. The contrast with the all-negative family shows why the infeasible-instance generator needs to be specified.

Table 3.4: Fresh mixed-sign stress-test results (additional family). Here d denotes density; times are means in milliseconds. Counts are primary projected-solver outcomes; all original LPs are infeasible.

m	d	Projector	Orig. (ms)	Proj. (ms)	Inf.	Opt.	Error
100	0.1	Sparse	0.68	2.96	3	7	0
100	0.1	Orthogonal	0.68	3.06	3	7	0
100	0.5	Sparse	1.06	2.38	10	0	0
100	0.5	Orthogonal	1.06	2.54	10	0	0
250	0.1	Sparse	1.55	29.61	9	1	0
250	0.1	Orthogonal	1.55	28.97	8	2	0
250	0.5	Sparse	4.65	16.25	10	0	0
250	0.5	Orthogonal	4.65	18.04	10	0	0
500	0.1	Sparse	4.51	178.10	9	0	1
500	0.1	Orthogonal	4.51	192.98	10	0	0
500	0.5	Sparse	18.90	86.32	9	0	1
500	0.5	Orthogonal	18.90	102.85	8	0	2

The four unresolved primary runs returned an unknown HiGHS model status. Separate diagnostic solves with HiGHS interior point classified all four as infeasible; disabling presolve for dual simplex did not resolve them. These checks are saved separately and do not replace the primary counts or enter the benchmark timings.

Across both infeasible families, the original LP was faster on average than constructing and solving its projected relaxation in every configuration. Projection adds sampling and matrix-product costs and usually makes the constraint matrix dense. It can also hide an immediate presolve contradiction, or produce a feasible relaxation. These results concern intentionally easy-to-certify infeasible instances and are not evidence that projection is slower for arbitrary infeasible LPs.

3.4 Verification and Reproducibility

Since $c = \mathbf{1}$ and $x \geq 0$, every feasible LP in the random benchmark is bounded below by zero. These runs do not test unboundedness. We therefore performed separate deterministic checks.

The counterexample in Theorem 2.2.1 was solved with original optimal value -2 and a projected unbounded status. For the forward implication, we used

$$A = \begin{pmatrix} 1 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad c = \begin{pmatrix} -1 \\ 0 \\ 0 \end{pmatrix}, \quad S = \begin{pmatrix} 1 & 1 \end{pmatrix}.$$

Here $x_0 = (1, 0, 1)^\top$ and $d = (1, 1, 0)^\top$ satisfy $Ax_0 = b$, $Ad = 0$, $d \geq 0$, and $c^\top d = -1$. Both original and projected LPs were reported unbounded. These checks illustrate the theorem; its proof establishes the implication for all admissible LPs.

An additional full-dimensional invertible projection returned the same optimal value as its original LP. We also checked the equality-dual sign convention, reduced-cost nonnegativity, and complementary slackness on that deterministic problem. These checks guard against implementation errors in the projected solve and lifted-dual recovery.

The new repository contains the independent implementation, pinned numerical dependencies, raw and aggregate results, source checksums, environment metadata, deterministic checks, and separate solver diagnostics. A verifier recomputes the aggregate statistics from the raw records and checks pairing, counts, provenance hashes, and diagnostic coverage. The report tables are generated from those aggregates and can be checked against the committed results. A rerun uses a new output directory to retain the recorded baseline. Timing values can change with hardware and system load.

3.5 Summary of the New Experiments

The fresh run comprises 300 projected LPs from 180 independent originals. Dual and primal recoveries yielded 0/60 and 0/60 accepted original-feasible candidates, respectively. Feasible-grid time ratios range from 1.07 to 4.07, measuring relaxation computation rather than successful original-LP solution. All-negative projections preserved infeasibility in 120/120 cases. The mixed-sign stress test produced 99 infeasible, 17 optimal and 4 unresolved error outcomes.

Chapter 4

Netlib Benchmark Extension

4.1 Scope and Reproducible Protocol

We extend the synthetic evaluation using five established LPs from the Netlib collection [12]. BLEND is an oil-refinery model variant; STOCFOR1 is a deterministic member of a forestry-planning family. AFIRO and ADLITTLE were supplied by Stanford SOL, and SCFXM1 belongs to the Ho–Loute staircase collection. These are historical application-derived or established solver benchmarks, not newly collected field data. The selection is a small, fixed benchmark suite, not a representative sample of all industrial LPs.

The source files are downloaded from a pinned COIN-OR mirror revision and checked by SHA-256 [13]. Each original row inequality is converted exactly to an equality with a non-negative slack or surplus variable. The selected models have nonnegative variables with no finite upper bounds. Minimization sense and zero objective offset are checked; unsupported model types are rejected. Both the native and standard-form LPs match Netlib’s published optima to the stated numerical tolerance. We do not add artificial variable bounds to force the projected LPs to remain bounded.

Table 4.1: Benchmark dimensions after exact standard-form conversion. Original storage is CSR; objective values are those recomputed by HiGHS.

LP	m	n	$\text{nnz}(A)$	CSR (KiB)	Optimum
ADLITTLE	56	138	424	5.19	225494.963162
AFIRO	27	51	102	1.30	-464.753143
BLEND	74	114	522	6.41	-30.812150
SCFXM1	330	600	2732	33.31	18416.759028
STOCFOR1	117	165	501	6.33	-41131.976219

For each of these five *fixed* LPs we use Gaussian, sparse-valued, and CountSketch projectors, $k = \lfloor \rho m \rfloor$ for $\rho \in \{0.5, 0.75\}$, and ten independent projector seeds. Thus there are 300 projected solves, but only five distinct original LPs. Variability within a configuration is over sketches, not independently sampled application problems. Projector distributions follow the preceding definitions. The master seed is 20261004; independent streams are derived from dataset, ratio, method, and repetition indices. Method execution order rotates across repetitions.

All timed solves use SciPy’s HiGHS dual-simplex method, presolve, feasibility tolerances 10^{-7} , a single requested thread, and a 30-second per-solve limit. The run used Python 3.12.14, NumPy 2.5.3, SciPy 1.18.1, and HiGHS 1.12.0 on macOS arm64, on 4 October 2026. An original solve is paired with each dataset/ratio/repetition and shared by its three projector comparisons. Projected time includes sampling, multiplication and sparse format conversion, and the solver call. Recovery is timed separately. Reading/converting MPS input, quality checks, and certificate solves are excluded from benchmark timings. Every group has ten observations; raw times, sample standard deviations, and conditional metric counts are supplied in the repository.

4.2 Solver Status and Original Feasibility

Because all five original LPs are feasible and bounded, their projected relaxations are feasible in exact arithmetic, but need not remain bounded. For each reported unbounded projection we separately solve for a nonnegative direction d satisfying

$$SAd = 0, \quad c^\top d = -1.$$

The stored numerical certificate must have $\|SAd\|_\infty \leq 10^{-6}$, $\min_i d_i \geq -10^{-7}$, and $|c^\top d + 1| \leq 10^{-6}$. Together with an original feasible point, this gives a numerical witness of projected unboundedness. The certificate is an untimed diagnostic, not part of the claimed workflow cost. Its original residual $\|Ad\|_\infty$ is also recorded.

Table 4.2: Projected status counts, ten sketches per cell pair. O: finite optimum; U: unbounded. No infeasible, limit, or error outcomes occurred.

LP	Projector	$\rho = .5$: O	$\rho = .5$: U	$\rho = .75$: O	$\rho = .75$: U
ADLITTLE	Gaussian	4	6	10	0
ADLITTLE	Sparse	3	7	10	0
ADLITTLE	CountSketch	0	10	1	9
AFIRO	Gaussian	1	9	9	1
AFIRO	Sparse	0	10	8	2
AFIRO	CountSketch	0	10	1	9
BLEND	Gaussian	0	10	8	2
BLEND	Sparse	0	10	9	1
BLEND	CountSketch	0	10	0	10
SCFXM1	Gaussian	0	10	10	0
SCFXM1	Sparse	0	10	10	0
SCFXM1	CountSketch	0	10	0	10
STOCFOR1	Gaussian	0	10	0	10
STOCFOR1	Sparse	0	10	0	10
STOCFOR1	CountSketch	0	10	0	10

Across 300 projected LPs, 216 are unbounded and 84 have finite optima. All 216 unbounded outcomes have passing numerical ray certificates; none of the stored directions satisfies the original equalities at the certificate tolerance. Of the 84 finite projected optima, 0 raw vectors pass original feasibility. Dual recovery encounters a singular selected basis in 84/84 attempts and produces no candidate. Primal recovery produces 84 candidates, all from rank-deficient selected systems (84/84); 0 pass original feasibility. These are failures of the specific recovery heuristics, not evidence that no feasible retrieval method exists.

The acceptance rule is the same as in the synthetic experiments: both the original equality residual and nonnegativity must pass. For raw projected vectors, the native MPS inequalities are checked as well; none passes. The two thesis recovery procedures are applied only when the projected LP has a finite optimum. A singular selected basis is reported as a recovery failure, not silently dropped from its attempts. An invertible, full-dimensional orthogonal

control ($k = m$) is also run on each LP: all five preserve the original optimum and feasibility. This isolates the effect of constraint reduction from a mere change of coordinates.

4.3 Time and Storage Trade-offs

Unlike the synthetic suite, the Netlib originals are stored as CSR matrices. Gaussian products are dense; sparse-valued and CountSketch products are stored in CSR. “Matrix storage” below counts the actual array buffers (values, indices, and row pointers for CSR), not peak process memory. Projector buffers are recorded separately in the raw data. The legacy recovery heuristics require a dense copy of A ; its storage is explicitly excluded from the projected-matrix comparison, so these figures are not total end-to-end memory savings.

Table 4.3: Mean time and projected-matrix storage at $\rho = 0.75$ (ten sketches per row). Original/projected times are milliseconds; projected totals include sampling and multiplication. Density is the percentage of nonzero matrix entries. All statuses are included.

LP	Projector	Orig. (ms)	Proj. (ms)	Density (%)	SA (KiB)
ADLITTLE	Gaussian	1.41	3.53	100.0	45.28
ADLITTLE	Sparse	1.41	3.55	61.0	41.58
ADLITTLE	CountSketch	1.41	1.21	7.0	4.92
AFIRO	Gaussian	0.77	0.91	100.0	7.97
AFIRO	Sparse	0.77	1.01	50.5	6.12
AFIRO	CountSketch	0.77	0.77	9.5	1.22
BLEND	Gaussian	1.50	4.58	100.0	48.98
BLEND	Sparse	1.50	4.01	64.8	47.82
BLEND	CountSketch	1.50	1.32	7.8	5.94
SCFXM1	Gaussian	4.89	968.90	100.0	1157.81
SCFXM1	Sparse	4.89	481.99	63.6	1105.41
SCFXM1	CountSketch	4.89	5.86	1.8	32.51
STOCFOR1	Gaussian	1.40	13.23	100.0	112.15
STOCFOR1	Sparse	1.40	11.62	60.5	102.04
STOCFOR1	CountSketch	1.40	1.52	3.4	6.12

The table reports means at $\rho = 0.75$; both ratios and all dispersion measures remain available in machine-readable form. Times include all statuses, including unbounded outcomes. A smaller or faster projected problem is not a faster solution of the original LP when it is unbounded or its recovered vector is infeasible. CountSketch can retain much

smaller matrix buffers, but in this experiment that storage benefit does not translate into an accepted original-feasible solution.

4.4 What Is Extended Relative to the Thesis?

Computational limitations were already discussed in the thesis, not discovered for the first time here. Its experimental discussion [5, pp. 38–40] identifies projection overhead, densification of SA , dense QR costs, and the efficiency of the original solver’s presolve. Table 4.1 also reports substantial negative mass even where equality residuals round to zero. The interior-point discussion on pp. 31–35 concerns time and memory costs in the literature, not a measured hardware limit in the author’s experiments. We found no explicit out-of-memory event or stated RAM cap explaining the largest tested size.

The new contribution to this course report is therefore an empirical and reproducibility extension: explicit acceptance counts, status separation, paired seeded experiments, application-related benchmarks, real sparse storage, CountSketch, two compression ratios, saved solution vectors, and checkable unbounded-ray certificates. It does not establish an asymptotic improvement or a new recovery algorithm. The original thesis prose about projected feasibility is stronger than its all-zero misclassification tables support; the new mixed-sign family tests that possibility separately, without rewriting the old evidence.

The benchmarks are deliberately modest in size and run in one software environment. They do not test large-scale memory exhaustion, provide a scaling law, or exhaust retrieval methods. Improved retrieval procedures exist, including the work of Liberti, Manca, and Poirion [11]; they are not evaluated here. All conclusions are restricted to the basic relaxation and the two stated thesis heuristics.

The repository supplies raw results, saved vectors, version pins and hashes. Its read-only verifier reconstructs sketches, checks metrics, certificates and summaries, and tests row conversion and determinism. Generated tables are checked against the results. Raw MPS files are fetched, not redistributed: the mirror’s build-system license excludes them. The synthetic baseline is preserved unchanged.

Chapter 5

Conclusion

This project explored matrix sketching for linear programming, using the author’s master’s thesis as its theoretical starting point. The Johnson–Lindenstrauss lemma motivates dimensionality reduction through distance preservation for finite sets. The retained unboundedness result follows from a different property: projecting equalities enlarges the feasible set. An original unbounded ray remains feasible, whereas the converse implication fails.

The revised synthetic evaluation did not yield valid recovered solutions under the two tested heuristics. All legacy all-negative projections remained infeasible, whereas the additional mixed-sign family produced some feasible relaxations and unresolved primary statuses. The instance distribution and recovery procedure therefore matter alongside running time.

The Netlib extension adds five fixed benchmarks, two compression ratios, and three projectors, including CountSketch. Of 300 projections, 216 are unbounded with verified numerical ray certificates, while 84 have finite optima; neither the raw nor the recovered candidates pass original feasibility. All five full-dimensional controls pass. The covariance and subspace-embedding discussion clarifies why geometric sketch quality alone does not imply validity of a compressed equality LP.

These small benchmarks use one solver environment and do not establish large-scale performance guarantees. Smaller matrix storage or faster relaxation computation is not a successful LP speedup without an accepted original-feasible solution. Other retrieval and sketch-based optimization methods remain outside this evaluation.

Bibliography

- [1] K. Vu, P.-L. Poirion, and L. Liberti. Random projections for linear programming. *Mathematics of Operations Research*, 43(4):1051–1071, 2018. doi:10.1287/moor.2017.0894.
- [2] S. Dasgupta and A. Gupta. An elementary proof of a theorem of Johnson and Lindenstrauss. *Random Structures & Algorithms*, 22(1):60–65, 2003. doi:10.1002/rsa.10073.
- [3] D. Achlioptas. Database-friendly random projections: Johnson–Lindenstrauss with binary coins. *Journal of Computer and System Sciences*, 66(4):671–687, 2003.
- [4] R. J. Vanderbei. *Linear Programming: Foundations and Extensions*. 5th edition, Springer, 2020.
- [5] J. Zhumabek. *Matrix Sketching and Linear Programming*. M.Sc. thesis in Applied Mathematics, Nazarbayev University, 2025.
- [6] J. Zhumabek. *Random_Projections_master_thesis*. Companion code for the master’s thesis. GitHub repository, 2025. https://github.com/Jamil997/Random_Projections_master_thesis. Inspected at commit 3d2ff96; rechecked 4 October 2026.
- [7] K. L. Clarkson and D. P. Woodruff. Low rank approximation and regression in input sparsity time. *Proceedings of STOC*, pp. 81–90, 2013. <https://arxiv.org/abs/1207.6365>.
- [8] N. Halko, P.-G. Martinsson, and J. A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011. <https://arxiv.org/abs/0909.4061>.

- [9] M. Ghashami, E. Liberty, J. M. Phillips, and D. P. Woodruff. Frequent Directions: Simple and deterministic matrix sketching. *SIAM Journal on Computing*, 45(5):1762–1792, 2016. <https://arxiv.org/abs/1501.01711>.
- [10] A. Chowdhury, G. Dexter, P. London, H. Avron, and P. Drineas. Faster randomized interior point methods for tall/wide linear programs. *Journal of Machine Learning Research*, 23(336):1–48, 2022. <https://jmlr.org/papers/v23/21-0709.html>.
- [11] L. Liberti, B. Manca, and P.-L. Poirion. Random projections for linear programming: An improved retrieval phase. *ACM Journal of Experimental Algorithmics*, 28, Article 2.2, 2023. doi:10.1145/3617506.
- [12] Netlib. Linear programming test problems: descriptions and reference optima. <https://www.netlib.org/lp/data/readme>. Accessed 4 October 2026.
- [13] COIN-OR. *Data-Netlib*, MPS benchmark mirror. <https://github.com/coin-or-tools/Data-Netlib>. Revision f1cc423067407d55d579c9c35fb01edf860dbc24.